```
Find suppliers whose number of products is
greater than the average number of products supplied per
supplier.

s1 --> c1
s2 --> c2
s3 --> c3

avg -> c1+c2+c3/3



select
  SupplierID,
  count(*) as no_of_products
from `e1.products`
group by SupplierID









select
  avg(no_of_products)
from (select
      SupplierID,
      count(*) as no_of_products
    from `e1.products`
    group by SupplierID
    )
```

```sql
select
  SupplierID,
  count(*) as no_of_prds
from `e1.products`
group by SupplierID
having no_of_prds >= (
                    select
                      avg(no_of_products)
                    from (select
                            SupplierID,
                            count(*) as no_of_products
                          from `e1.products`
                          group by SupplierID
                          )
                    )
```

"Show me the product name and the category name."

products -- Product Name
linkage: categoryID
Category -- category Name

```
tableName.columnName

syntax:

select
from table1
join table2
on table1.key = table2.key


"Show me the product name and the category name."

select *
from `e1.products`
limit 5


select *
from `e1.categories`
limit 5



alter table `e1.categories`
rename column string_field_0 to CategoryID
```

```sql
alter table `e1.categories`
rename column string_field_1 to CategoryName



select
  p.ProductID,
  p.ProductName,
  c.CategoryID,
  c.CategoryName
from `e1.categories` as c
inner join `e1.products` as p
on c.CategoryID = p.CategoryID




Display products belonging to the Electronics category.

select
  p.ProductID,
  p.ProductName,
  c.CategoryID,
  c.CategoryName
from `e1.categories` as c
inner join `e1.products` as p
on c.CategoryID = p.CategoryID
where c.CategoryName= 'Electronics'
```

```
Display products with their category names, sorted by MRP from
highest to lowest.

select
  p.ProductID,
  p.ProductName,
  c.CategoryID,
  c.CategoryName
from `e1.categories` as c
inner join `e1.products` as p
on c.CategoryID = p.CategoryID
order by p.MRP desc


Display ProductName, CategoryName and discount amount.

select
  p.ProductID,
  p.ProductName,
  c.CategoryID,
  c.CategoryName,
  p.MRP - p.SellingPrice as discount_amt
from `e1.categories` as c
inner join `e1.products` as p
on c.CategoryID = p.CategoryID


Display each product along with its supplier name and supplier
state.
```

```sql
select *
from `e1.products`
limit 5


select *
from `e1.suppliers`
limit 5


select
  p.ProductName,
  s.SupplierID,
  s.SupplierName,
  s.State
from `e1.suppliers` as s
inner join `e1.products` as p
on p.SupplierID = s.SupplierID



Display all Cancelled customer orders along with customer name
and city.

select *
from `e1.customers`
limit 5

select *
from `e1.orders`
```

```
limit 5



select
  o.Status,
  c.Name,
  c.City
from `e1.orders` as o
inner join `e1.customers` as c
on o.CustomerID = c.CustomerID
where o.Status = "Cancelled"




Calculate total sales value for every order.

select *
from `e1.orders`
limit 5


select *
from `e1.order_items`
limit 5


select
  o.OrderID,
  sum(oi.Total) as total_sales
```

```
from `e1.orders` as o
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by o.OrderID



Calculate total sales value generated by each customer.

select

from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID


select *
from `e1.customers`
limit 5


select *
from `e1.order_items`
limit 5



1.customer name -> customer
2. linkage -> table: orders
customerid -> orders + customer
```

```
orderid -> orders + order_items
3.total sales -> order_items



select
  c.CustomerID,
  sum(oi.Total) as total_sales
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by c.CustomerID
order by total_sales desc



DPP Questions:

1. Product-Wise Sales With Product Name

2. CategoryName-Wise total Sales

3. Calculate total sales generated by products from each
supplier.

4. Display all products and any order-item sales associated with
them.
```

5. Find Products With No Sales

6. Customers With No Orders

7. For every customer, show the number of orders they have placed, including customers with zero orders.